Python Training Series

Module 12

Data Visualization with Matplotlib

Instructor Guide & Student Reference

How to use this document

This document serves dual purpose — as a live teaching guide during class and as a self-contained post-training reference. Each section presents concept explanations followed by complete, runnable code examples. Instructors should walk through each example in a Jupyter notebook or VS Code. Students can revisit any section independently after training.

12.1 Introduction to Matplotlib

12.1.1 What is Matplotlib?

Matplotlib is the foundational plotting library for Python. Created by John D. Hunter in 2003 to replicate MATLAB-style plotting, it has become the backbone of scientific visualization in Python. Almost every other visualization library — Seaborn, Pandas plot, Plotly's static export — either wraps or extends Matplotlib under the hood.

12.1.2 Matplotlib Architecture — Figure, Axes, Artist Hierarchy

Understanding the internal hierarchy is the single most important concept in Matplotlib. Everything you see in a plot is an Artist object organized in a strict tree:

Object	Description
`Figure`	The top-level container — the entire window or page. One Figure can hold many Axes. Think of it as the canvas.
`Axes`	A single plot area with its own coordinate system, title, x-axis and y-axis. This is where data actually gets drawn. Don't confuse with Axis (see below).
`Axis`	The individual x or y number line inside an Axes. Controls tick marks, tick labels, and limits.
`Artist`	Base class for everything drawn on the Figure — lines, text, patches, images, legends. When you call ax.plot(), it returns a Line2D artist.

12.1.3 pyplot vs Object-Oriented Interface

Matplotlib exposes two usage patterns. Understanding when to use each prevents common bugs and messy code in production.

pyplot (plt) Interface

State-machine style. Matplotlib tracks the 'current' figure and axes implicitly. Good for quick exploratory plots in notebooks.

Use when:

- Quick one-off exploration
- Single plot in a notebook cell
- Teaching / prototyping

Object-Oriented (OO) Interface

Explicit references to Figure and Axes objects. More verbose but clearer and safer, especially with multiple subplots or functions.

Use when:

- Multiple subplots on one figure
- Reusable plot functions / classes
- Production dashboards and reports

```
# ── pyplot (implicit) style ──────────────────────────
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot([1, 2, 3], [10, 20, 15])
plt.title('Sales Trend')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.show()

# ── Object-Oriented (explicit) style ─────────────────
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))    # fig = Figure,  ax = Axes
ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title('Sales Trend')
ax.set_xlabel('Month')
ax.set_ylabel('Revenue')
plt.show()
```

Note: The OO style is preferred in this course. It makes code easier to debug and extend.

12.1.4 Installation & Backends

```
# Install Matplotlib (and Pandas for later integration)
pip install matplotlib pandas

# Verify installation
python -c "import matplotlib; print(matplotlib.__version__)"

# In Jupyter — display inline (no plt.show() needed)
%matplotlib inline

# Interactive widgets in Jupyter Lab
%matplotlib widget
```

A backend is the rendering engine Matplotlib uses to draw to screen or file. Common backends:

- Agg — raster (PNG) rendering, no display window. Default for scripts and servers.
- TkAgg / Qt5Agg — interactive window for desktop use.
- inline — used by Jupyter notebooks to embed images in cells.
- widget — enables zoom/pan interaction inside Jupyter Lab.

12.2 Basic Plots

12.2.1 Line Plots

Line plots are the most common chart type for showing trends over time or continuous data. The ax.plot() method connects data points with straight line segments.

Single and Multiple Lines

```
import matplotlib.pyplot as plt
import numpy as np

months = np.arange(1, 13)                    # Jan-Dec
revenue_A = [45, 52, 49, 60, 72, 68, 75, 80, 77, 85, 90, 95]
revenue_B = [30, 38, 42, 50, 55, 60, 58, 65, 70, 68, 74, 80]

fig, ax = plt.subplots(figsize=(10, 5))

# Plot line A — solid blue with circle markers
ax.plot(months, revenue_A, color='steelblue', linewidth=2,
        marker='o', markersize=6, label='Product A')

# Plot line B — dashed orange with square markers
ax.plot(months, revenue_B, color='darkorange', linewidth=2,
        linestyle='--', marker='s', markersize=6, label='Product B')

ax.set_title('Monthly Revenue Comparison', fontsize=16, fontweight='bold')
ax.set_xlabel('Month')
ax.set_ylabel('Revenue (₹ Lakhs)')
ax.set_xticks(months)
ax.set_xticklabels(['Jan','Feb','Mar','Apr','May','Jun',
                    'Jul','Aug','Sep','Oct','Nov','Dec'])
ax.legend()
ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

> **Tip:** Pass label= to each plot() call, then call ax.legend() once at the end. Matplotlib auto-assigns colors if you don't specify them.

12.2.2 Scatter Plots

Scatter plots reveal relationships between two numeric variables. You can encode a third dimension through marker size (s) and a fourth through color (c).

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
age        = np.random.randint(22, 60, 100)
salary     = age * 1500 + np.random.normal(0, 10000, 100)
experience = np.random.randint(1, 30, 100)
dept       = np.random.choice(['Sales','Tech','HR'], 100)

colors = {'Sales': 'steelblue', 'Tech': 'darkorange', 'HR': 'seagreen'}
color_vals = [colors[d] for d in dept]

fig, ax = plt.subplots(figsize=(9, 6))

sc = ax.scatter(
    age, salary,
    s = experience * 8,      # marker SIZE encodes experience
    c = color_vals,          # color encodes department
    alpha = 0.7,             # transparency to handle overlap
    edgecolors = 'white',
    linewidths = 0.5
)

ax.set_title('Age vs Salary  (bubble size = experience)', fontsize=14)
ax.set_xlabel('Age')
ax.set_ylabel('Annual Salary (₹)')

# Manual legend for departments
from matplotlib.patches import Patch
legend_els = [Patch(color=v, label=k) for k, v in colors.items()]
ax.legend(handles=legend_els, title='Department')

plt.tight_layout()
plt.show()
```

12.2.3 Bar Charts

Bar charts compare categorical data. Matplotlib supports vertical, horizontal, grouped, and stacked variants.

Vertical and Horizontal Bars

```
import matplotlib.pyplot as plt
import numpy as np
```

```
products = ['Laptop', 'Phone', 'Tablet', 'Watch', 'Earbuds']
sales    = [320, 540, 210, 380, 460]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Vertical bar
bars = ax1.bar(products, sales, color='steelblue', edgecolor='white',
width=0.6)
ax1.set_title('Product Sales — Vertical')
ax1.set_ylabel('Units Sold')
# Add value labels on top of each bar
for bar in bars:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
             str(int(bar.get_height())), ha='center', va='bottom',
fontsize=10)

# Horizontal bar (good for long category names)
ax2.barh(products, sales, color='darkorange', edgecolor='white')
ax2.set_title('Product Sales — Horizontal')
ax2.set_xlabel('Units Sold')
ax2.invert_yaxis()   # Highest value at top

plt.tight_layout()
plt.show()
```

Grouped and Stacked Bar Charts

```
import matplotlib.pyplot as plt
import numpy as np

quarters = ['Q1', 'Q2', 'Q3', 'Q4']
online   = [120, 145, 160, 200]
offline  = [200, 180, 170, 150]
x = np.arange(len(quarters))
w = 0.35

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# ── Grouped ──────────────────────────────────────
ax1.bar(x - w/2, online,  w, label='Online',  color='steelblue')
ax1.bar(x + w/2, offline, w, label='Offline', color='darkorange')
ax1.set_xticks(x)
ax1.set_xticklabels(quarters)
ax1.set_title('Grouped Bar Chart')
ax1.legend()

# ── Stacked ──────────────────────────────────────
ax2.bar(quarters, online,  label='Online',  color='steelblue')
ax2.bar(quarters, offline, label='Offline', color='darkorange',
        bottom=online)              # <-- key parameter
```

```
ax2.set_title('Stacked Bar Chart')
ax2.legend()

plt.tight_layout()
plt.show()
```

12.2.4 Histograms

Histograms show the frequency distribution of a numeric variable. The choice of bin count significantly affects what the chart reveals.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
scores = np.random.normal(loc=70, scale=12, size=500)   # Exam scores

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Standard histogram
axes[0].hist(scores, bins=20, color='steelblue', edgecolor='white')
axes[0].set_title('Standard (bins=20)')

# Density (probability density function)
axes[1].hist(scores, bins=20, density=True, color='darkorange',
edgecolor='white')
axes[1].set_title('Density=True')

# Cumulative histogram
axes[2].hist(scores, bins=20, cumulative=True, color='seagreen',
edgecolor='white')
axes[2].set_title('Cumulative')

for ax in axes:
    ax.set_xlabel('Score')

plt.suptitle('Exam Score Distribution', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

> **Tip:** Use numpy's histogram_bin_edges() or 'auto' / 'fd' / 'sturges' as the bins argument to let Matplotlib pick a sensible count automatically.

12.2.5 Pie Charts

Pie charts work best when you have 5 or fewer categories and want to show part-to-whole relationships. For many categories, use a bar chart instead.

```
import matplotlib.pyplot as plt

channels = ['Direct', 'Social Media', 'Email', 'Referral', 'Organic']
sizes    = [30, 25, 20, 15, 10]
explode  = (0.05, 0.1, 0.05, 0, 0)   # Pull out 'Social Media'
colors   = ['#2196F3', '#FF9800', '#4CAF50', '#9C27B0', '#F44336']

fig, ax = plt.subplots(figsize=(8, 6))
wedges, texts, autotexts = ax.pie(
    sizes,
    labels    = channels,
    autopct   = '%1.1f%%',     # Show percentage with 1 decimal
    explode   = explode,
    colors    = colors,
    shadow    = True,
    startangle = 90            # Start from top
)

# Style the percentage text
for autotext in autotexts:
    autotext.set_fontsize(10)
    autotext.set_color('white')
    autotext.set_fontweight('bold')

ax.set_title('Lead Source Distribution', fontsize=15, fontweight='bold')
plt.show()
```

12.2.6 Box Plots and Violin Plots

Box plots (also called box-and-whisker plots) summarize a distribution in five statistics: minimum, Q1 (25th percentile), median, Q3 (75th percentile), and maximum. Outliers appear as individual points beyond the whiskers. Violin plots add a kernel density estimate, showing the full shape of the distribution alongside the box plot summary.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)
data = {
    'North' : np.random.normal(60, 10, 100),
    'South' : np.random.normal(55, 15, 100),
    'East'  : np.random.normal(70, 8,  100),
```

```
    'West'  : np.random.normal(65, 12, 100),
}

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# ── Box Plot ──────────────────────────────────────────
ax1.boxplot(data.values(), labels=data.keys(),
            patch_artist=True,           # filled boxes
            boxprops=dict(facecolor='steelblue', alpha=0.7),
            medianprops=dict(color='red', linewidth=2),
            flierprops=dict(marker='o', color='grey', alpha=0.5))
ax1.set_title('Box Plot — Regional Sales')
ax1.set_ylabel('Sales (Units)')

# ── Violin Plot ───────────────────────────────────────
parts = ax2.violinplot(data.values(), showmeans=True, showmedians=True)
ax2.set_xticks(range(1, len(data)+1))
ax2.set_xticklabels(data.keys())
ax2.set_title('Violin Plot — Regional Sales')
ax2.set_ylabel('Sales (Units)')

plt.suptitle('Distribution of Regional Sales', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()
```

Note: Violin plots are more informative than box plots for bimodal or skewed distributions but are less familiar to non-technical audiences.

12.3 Customization and Styling

12.3.1 Colors

Matplotlib supports multiple ways to specify colors. Choose the most readable option for your context:

```
# ── Named colors ──────────────────────────────────────
ax.plot(x, y, color='steelblue')          # CSS color name
ax.plot(x, y, color='tab:orange')         # Tableau palette

# ── Hex codes ─────────────────────────────────────────
ax.plot(x, y, color='#2E75B6')            # 6-digit hex
ax.plot(x, y, color='#2E75B680')          # hex + alpha

# ── RGB / RGBA tuples ─────────────────────────────────
ax.plot(x, y, color=(0.18, 0.46, 0.71))          # RGB  [0-1]
ax.plot(x, y, color=(0.18, 0.46, 0.71, 0.5))     # RGBA [0-1]

# ── Colormaps ─────────────────────────────────────────
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 50)
cmap = plt.colormaps['viridis']    # Load a colormap
for i in range(5):
    color = cmap(i / 5)            # Sample color from map
    ax.plot(x, np.sin(x + i), color=color, label=f'Line {i+1}')

# Show available colormaps
# plt.colormaps()  →  lists all registered colormaps
```

12.3.2 Markers and Line Styles

Marker Code	Shape	Line Style	Effect
`'o'`	Circle	`'-'`	Solid
`'s'`	Square	`'--'`	Dashed
`'^'`	Triangle up	`'-.'`	Dash-dot
`'D'`	Diamond	`':'`	Dotted
`'+'`	Plus	`'none'`	No line
`'x'`	X cross		

12.3.3 Labels, Titles, and Annotations

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(x, y, color='steelblue', linewidth=2)

# Title and axis labels with font control
ax.set_title('Sine Wave', fontsize=18, fontweight='bold', pad=15)
ax.set_xlabel('Angle (radians)', fontsize=13)
ax.set_ylabel('Amplitude',        fontsize=13)

# ── Annotation with arrow ──────────────────────────
ax.annotate(
    text     = 'Peak',
    xy       = (np.pi/2, 1),       # Point to annotate
    xytext   = (np.pi/2 + 0.8, 0.8),  # Text location
    arrowprops = dict(arrowstyle='->', color='red', lw=1.5),
    fontsize = 12,
    color    = 'red'
)

# ── Horizontal / Vertical reference lines ──────────
ax.axhline(0, color='grey', linewidth=0.8, linestyle='--')
ax.axvline(np.pi, color='orange', linewidth=1, linestyle=':')

# ── Text label at a point ──────────────────────────
ax.text(np.pi + 0.05, 0.05, 'x = π', color='darkorange', fontsize=11)

plt.tight_layout()
plt.show()
```

12.3.4 Legends

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(9, 5))
x = np.linspace(0, 10, 100)

l1, = ax.plot(x, np.sin(x), label='sin(x)')
l2, = ax.plot(x, np.cos(x), label='cos(x)')
l3, = ax.plot(x, np.sin(x)*np.cos(x), label='sin·cos')

# Automatic placement
ax.legend()                         # Matplotlib picks best loc
```

```
# Manual placement
ax.legend(loc='upper right')        # String location
ax.legend(loc=(0.7, 0.6))           # (x,y) in axes coords 0-1
ax.legend(bbox_to_anchor=(1, 1), loc='upper left')  # Outside axes

# Customization
ax.legend(
    handles=[l1, l2],               # Only specific artists
    labels =['Sine', 'Cosine'],     # Override labels
    title  ='Functions',
    fontsize=11,
    framealpha=0.8,
    ncol=2,                         # Two-column legend
    shadow=True
)
plt.show()
```

12.3.5 Grid Lines and Spines

```
fig, ax = plt.subplots(figsize=(8, 5))

# Grid — major and minor
ax.grid(True, which='major', linestyle='--', linewidth=0.8, alpha=0.7)
ax.minorticks_on()
ax.grid(True, which='minor', linestyle=':', linewidth=0.4, alpha=0.4)

# Spines — the four border lines of the plot
ax.spines['top'].set_visible(False)     # Remove top spine
ax.spines['right'].set_visible(False)   # Remove right spine
ax.spines['left'].set_color('steelblue')
ax.spines['bottom'].set_linewidth(1.5)

plt.show()
```

12.4 Subplots and Layout

12.4.1 plt.subplots() — Grid of Axes

plt.subplots(nrows, ncols) returns a Figure and a 2D array of Axes. This is the most common way to create multi-panel figures.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 200)

# 2×2 grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# axes is a 2D numpy array; index with [row, col]
axes[0, 0].plot(x, np.sin(x),    color='steelblue')
axes[0, 0].set_title('sin(x)')

axes[0, 1].plot(x, np.cos(x),    color='darkorange')
axes[0, 1].set_title('cos(x)')

axes[1, 0].plot(x, np.sin(2*x),  color='seagreen')
axes[1, 0].set_title('sin(2x)')

axes[1, 1].plot(x, np.cos(2*x),  color='crimson')
axes[1, 1].set_title('cos(2x)')

# Flatten to iterate easily
for ax in axes.flat:
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.axhline(0, color='grey', linewidth=0.5)

fig.suptitle('Trigonometric Functions', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

12.4.2 GridSpec — Complex Asymmetric Layouts

GridSpec lets you span rows and columns, creating dashboards where plots have different sizes. This is the go-to for publication-quality multi-panel figures.

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

```
import numpy as np

fig = plt.figure(figsize=(14, 8))

# Define a 3×3 grid with spacing
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.4, wspace=0.4)

# Large plot spanning top 2 rows and all 3 columns
ax_main = fig.add_subplot(gs[0:2, :])   # rows 0–1, all cols

# Three small plots in the bottom row
ax_bl = fig.add_subplot(gs[2, 0])       # bottom-left
ax_bc = fig.add_subplot(gs[2, 1])       # bottom-center
ax_br = fig.add_subplot(gs[2, 2])       # bottom-right

x = np.linspace(0, 10, 200)
ax_main.plot(x, np.sin(x) * np.exp(-x/10), color='steelblue', lw=2)
ax_main.set_title('Main Chart (Spans Full Width)', fontsize=14)

ax_bl.bar(['A','B','C'], [3,7,5], color='darkorange')
ax_bl.set_title('Q1')
ax_bc.bar(['A','B','C'], [6,4,8], color='seagreen')
ax_bc.set_title('Q2')
ax_br.bar(['A','B','C'], [2,9,6], color='crimson')
ax_br.set_title('Q3')

fig.suptitle('Dashboard Layout with GridSpec', fontsize=16,
fontweight='bold')
plt.show()
```

12.4.3 Sharing Axes Between Subplots

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 200)

# sharex=True: all axes share the same x-axis zoom/pan
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

axes[0].plot(x, np.sin(x),          color='steelblue')
axes[0].set_ylabel('sin(x)')

axes[1].plot(x, np.sin(x)**2,       color='darkorange')
axes[1].set_ylabel('sin²(x)')

axes[2].plot(x, np.cumsum(np.sin(x)) * 0.05, color='seagreen')
axes[2].set_ylabel('Cumulative')
axes[2].set_xlabel('x')
```

```
# Remove x-tick labels from upper plots (sharex handles alignment)
for ax in axes[:-1]:
    plt.setp(ax.get_xticklabels(), visible=False)

fig.suptitle('Shared X-Axis — Zooming one zooms all', fontsize=13)
plt.tight_layout()
plt.show()
```

12.4.4 tight_layout() and constrained_layout

By default Matplotlib does not prevent subplot labels from overlapping. Both tight_layout() and constrained_layout automatically adjust spacing.

```
# Method 1: tight_layout  (post-draw adjustment)
plt.tight_layout(pad=1.5)         # pad in inches
plt.tight_layout(rect=[0,0,1,0.95])  # Leave space for suptitle

# Method 2: constrained_layout  (preferred for complex grids)
fig, axes = plt.subplots(2, 2, figsize=(10, 8),
                         constrained_layout=True)

# Or enable globally in rcParams:
plt.rcParams['figure.constrained_layout.use'] = True
```

12.5 Advanced Visualization

12.5.1 3D Plots with mpl_toolkits.mplot3d

Matplotlib includes a 3D toolkit for surface, scatter, and line plots in three dimensions. The key step is creating an Axes3D by passing projection='3d' to add_subplot.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D   # Required import
import numpy as np

fig = plt.figure(figsize=(16, 5))

# ── 3D Line ──────────────────────────────────────────
ax1 = fig.add_subplot(131, projection='3d')
t = np.linspace(0, 4*np.pi, 200)
ax1.plot(np.cos(t), np.sin(t), t, color='steelblue')
ax1.set_title('3D Helix')

# ── 3D Scatter ───────────────────────────────────────
ax2 = fig.add_subplot(132, projection='3d')
np.random.seed(0)
x = np.random.randn(100); y = np.random.randn(100); z =
np.random.randn(100)
ax2.scatter(x, y, z, c=z, cmap='viridis', alpha=0.8, s=30)
ax2.set_title('3D Scatter')

# ── 3D Surface ───────────────────────────────────────
ax3 = fig.add_subplot(133, projection='3d')
X = np.linspace(-3, 3, 60)
Y = np.linspace(-3, 3, 60)
X, Y = np.meshgrid(X, Y)
Z = np.sin(np.sqrt(X**2 + Y**2))
surf = ax3.plot_surface(X, Y, Z, cmap='plasma', alpha=0.9)
fig.colorbar(surf, ax=ax3, shrink=0.5)
ax3.set_title('3D Surface')

plt.tight_layout()
plt.show()
```

12.5.2 Heatmaps with imshow() and pcolormesh()

Heatmaps visualize matrix data where cell color encodes value. imshow() is best for regular grids; pcolormesh() for non-uniform grids.

```
import matplotlib.pyplot as plt
import numpy as np

# Correlation matrix example
labels = ['Revenue', 'Units', 'Discount', 'Returns', 'Profit']
data = np.array([
    [ 1.00,  0.85, -0.30, -0.55,  0.92],
    [ 0.85,  1.00, -0.10, -0.40,  0.78],
    [-0.30, -0.10,  1.00,  0.25, -0.45],
    [-0.55, -0.40,  0.25,  1.00, -0.60],
    [ 0.92,  0.78, -0.45, -0.60,  1.00],
])

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(data, cmap='RdYlGn', vmin=-1, vmax=1)

# Tick labels
ax.set_xticks(range(len(labels))); ax.set_xticklabels(labels, rotation=45,
ha='right')
ax.set_yticks(range(len(labels))); ax.set_yticklabels(labels)

# Annotate each cell with the correlation value
for i in range(len(labels)):
    for j in range(len(labels)):
        ax.text(j, i, f'{data[i,j]:.2f}',
                ha='center', va='center', fontsize=10,
                color='black' if abs(data[i,j]) < 0.7 else 'white')

fig.colorbar(im, ax=ax, label='Correlation')
ax.set_title('Correlation Heatmap', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

12.5.3 Contour Plots

```
import matplotlib.pyplot as plt
import numpy as np

X = np.linspace(-4, 4, 200)
Y = np.linspace(-4, 4, 200)
X, Y = np.meshgrid(X, Y)
Z = np.sin(X) * np.cos(Y) * np.exp(-(X**2 + Y**2)/10)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# contour — lines only
cs = ax1.contour(X, Y, Z, levels=15, cmap='RdBu')
ax1.clabel(cs, inline=True, fontsize=8)    # Label contour lines
ax1.set_title('contour()  — Lines')
```

```
# contourf — filled regions
cf = ax2.contourf(X, Y, Z, levels=20, cmap='RdBu')
fig.colorbar(cf, ax=ax2)
ax2.set_title('contourf() — Filled')

plt.tight_layout()
plt.show()
```

12.5.4 Animations with FuncAnimation

FuncAnimation repeatedly calls an update function, replacing artists on each frame. The result can be saved as GIF or MP4, or displayed live in a Jupyter notebook.

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 2*np.pi, 200)
line, = ax.plot(x, np.sin(x), color='steelblue', lw=2)

ax.set_ylim(-1.5, 1.5)
ax.grid(True, linestyle='--', alpha=0.5)
ax.set_title('Animated Sine Wave')

def update(frame):
    line.set_ydata(np.sin(x + frame * 0.1))    # Shift phase
    return line,

ani = animation.FuncAnimation(
    fig,
    update,
    frames=100,         # Number of frames
    interval=50,        # ms between frames
    blit=True           # Only redraw changed artists (faster)
)

# Save as GIF (requires Pillow) or MP4 (requires ffmpeg)
ani.save('sine_wave.gif', writer='pillow', fps=20)
# ani.save('sine_wave.mp4', writer='ffmpeg', fps=20)

plt.show()
```

12.5.5 Interactive Plots with %matplotlib widget

```
# Run this in Jupyter Lab (requires ipympl package)
# pip install ipympl

%matplotlib widget

import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(9, 5))
x = np.linspace(0, 10, 500)
ax.plot(x, np.sin(x))
ax.set_title('Interactive — zoom, pan, and save from toolbar')
plt.show()

# After %matplotlib widget, all figures are interactive.
# Use %matplotlib inline to switch back to static images.
```

12.6 Integration with Pandas and Polars

12.6.1 Plotting Directly from Pandas DataFrames

Pandas DataFrames and Series have a .plot() accessor that wraps Matplotlib. This is the fastest route from data to chart in exploratory analysis. All standard plot types are available via the kind= parameter.

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample sales dataset
df = pd.DataFrame({
    'Month'   : ['Jan','Feb','Mar','Apr','May','Jun'],
    'Online'  : [120, 145, 160, 200, 180, 210],
    'Offline' : [200, 180, 170, 150, 160, 140],
})
df.set_index('Month', inplace=True)

# ── Line plot ──────────────────────────────────────
ax = df.plot(kind='line', figsize=(9, 4), marker='o',
             title='Monthly Channel Sales')
ax.set_ylabel('Units Sold')
plt.tight_layout()
plt.show()

# ── Bar plot ───────────────────────────────────────
ax = df.plot(kind='bar', figsize=(9, 4), rot=0,
             title='Monthly Channel Sales')
ax.set_ylabel('Units Sold')
plt.tight_layout()
plt.show()

# ── Histogram of a single column ──────────────────
df['Online'].plot(kind='hist', bins=10, color='steelblue',
                  edgecolor='white', title='Online Sales Distribution')
plt.show()

# ── Scatter – two columns ─────────────────────────
df.plot(kind='scatter', x='Online', y='Offline',
        color='darkorange', s=80, figsize=(6, 5))
plt.show()
```

Note: df.plot() returns an Axes object. You can chain all standard Matplotlib customization on top — ax.set_xlabel(), ax.legend(), etc.

12.6.2 Converting Polars to Pandas for Plotting

Polars is a high-performance DataFrame library that does not natively integrate with Matplotlib's .plot() accessor. The standard workflow is to convert to Pandas for visualization.

```
import polars as pl
import matplotlib.pyplot as plt

# Create a Polars DataFrame
df_pl = pl.DataFrame({
    'region'  : ['North', 'South', 'East', 'West'],
    'revenue' : [450, 320, 580, 390],
    'cost'    : [310, 220, 400, 270],
})

# ── Option 1: Convert entire DataFrame ──────────────────
df_pd = df_pl.to_pandas()
df_pd.set_index('region')[['revenue','cost']].plot(
    kind='bar', figsize=(8,5), rot=0, title='Regional P&L'
)
plt.tight_layout()
plt.show()

# ── Option 2: Extract individual series ─────────────────
# (Avoids converting unused columns — good for wide DataFrames)
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(
    df_pl['region'].to_list(),    # Polars series → Python list
    df_pl['revenue'].to_list(),
    color='steelblue', label='Revenue'
)
ax.set_title('Regional Revenue')
ax.legend()
plt.tight_layout()
plt.show()
```

12.6.3 Best Practices for Large-Data Visualization

- Aggregate before plotting: Use Pandas groupby() or Polars group_by() to reduce millions of rows to meaningful summaries. Never plot raw big data directly — it overloads the renderer and creates unreadable charts.
- Use rasterization for scatter plots with 10k+ points: ax.scatter(..., rasterized=True) renders the scatter as a bitmap inside a vector PDF, keeping file size manageable.
- Consider datashader for very large datasets: When you have millions of points, datashader renders density maps that reveal true distribution without overplotting.
- Sample strategically: If you must plot individual points, sample with df.sample(n=5000) rather than plotting everything. Annotate the chart with the total record count.
- Use hex bins (ax.hexbin) instead of scatter for dense data: Hexbin aggregates points into hexagonal cells colored by count, revealing density without overplotting.

12.7 Publication-Ready Figures

12.7.1 DPI and Figure Size

The physical appearance of a Matplotlib figure is controlled by two parameters: figsize (width, height in inches) and dpi (dots per inch). Together they determine the pixel resolution of the output.

```
import matplotlib.pyplot as plt

# For screen / web  — 100 DPI is usually fine
fig, ax = plt.subplots(figsize=(10, 6), dpi=100)
# → 1000 × 600 pixels

# For print / journal — use 300 DPI minimum
fig, ax = plt.subplots(figsize=(7, 5), dpi=300)
# → 2100 × 1500 pixels

# Rule of thumb for academic journals:
#   Single column  → figsize=(3.5, 2.5), dpi=300
#   Double column  → figsize=(7.0, 4.5), dpi=300

# Check current rcParams defaults
print(plt.rcParams['figure.figsize'])   # [6.4, 4.8]
print(plt.rcParams['figure.dpi'])       # 100
```

12.7.2 Exporting Figures

```
fig.savefig('chart.png',  dpi=300, bbox_inches='tight')
fig.savefig('chart.pdf',  bbox_inches='tight')   # Vector — scales
perfectly
fig.savefig('chart.svg',  bbox_inches='tight')   # Vector — editable in
Inkscape
fig.savefig('chart.eps',  bbox_inches='tight')   # Required by some
journals

# bbox_inches='tight' trims whitespace and ensures legend/title
# are not clipped outside the canvas

# Transparent background (useful for slides)
fig.savefig('chart.png', dpi=150, transparent=True)
```

> **Tip:** Always use PDF or SVG for presentations and Word documents. PNG is fine for web, but re-exporting from PNG degrades quality. Vector formats scale losslessly to any size.

12.7.3 rcParams and Style Sheets

rcParams (runtime configuration parameters) are a dictionary that controls every default Matplotlib setting — font sizes, line widths, colors, figure size, etc. Style sheets bundle rcParam overrides into named themes.

```
import matplotlib.pyplot as plt

# ── Applying a built-in style sheet ──────────────────────
plt.style.use('seaborn-v0_8')        # Clean, modern look
plt.style.use('ggplot')              # R ggplot2 aesthetic
plt.style.use('bmh')                 # Bayesian Methods for Hackers
plt.style.use('dark_background')     # Dark theme for presentations

# Combine styles (right overrides left)
plt.style.use(['seaborn-v0_8', 'seaborn-v0_8-paper'])

# List all available styles
print(plt.style.available)

# ── Modifying individual rcParams ────────────────────────
plt.rcParams['font.family']      = 'Arial'
plt.rcParams['font.size']        = 12
plt.rcParams['axes.titlesize']   = 15
plt.rcParams['axes.labelsize']   = 12
plt.rcParams['xtick.labelsize']  = 10
plt.rcParams['ytick.labelsize']  = 10
plt.rcParams['lines.linewidth']  = 2
plt.rcParams['figure.figsize']   = (9, 6)

# ── Temporary style (context manager) ────────────────────
with plt.style.context('ggplot'):
    fig, ax = plt.subplots()
    ax.plot([1,2,3], [4,5,6])
    plt.show()
# rcParams restored after the with block

# ── Reset to defaults ────────────────────────────────────
plt.rcdefaults()
```

12.7.4 Colorblind-Friendly Palettes

Approximately 8% of men and 0.5% of women have red-green color blindness. Use palette choices that remain distinguishable when colors are perceived differently.

```
import matplotlib.pyplot as plt
import numpy as np

# ── Wong palette (8 colors, designed for colorblindness) ─
WONG = [
    '#000000',  # Black
    '#E69F00',  # Orange
    '#56B4E9',  # Sky Blue
    '#009E73',  # Green
    '#F0E442',  # Yellow
    '#0072B2',  # Blue
    '#D55E00',  # Vermilion
    '#CC79A7',  # Reddish Purple
]

# ── IBM Design colorblind-safe palette ──────────────────
IBM = ['#648FFF', '#785EF0', '#DC267F', '#FE6100', '#FFB000']

# ── Use with Matplotlib ─────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 5))
x = np.linspace(0, 10, 100)

for i, color in enumerate(WONG[:4]):
    ax.plot(x, np.sin(x + i), color=color,
            linewidth=2, label=f'Series {i+1}')

ax.legend()
ax.set_title('Using the Wong Colorblind-Friendly Palette')
plt.tight_layout()
plt.show()

# ── Diverging colormaps good for colorblindness ─────────
#   RdYlBu, PiYG, PRGn  →  distinguishable with most deficiencies
#   viridis, plasma, cividis  →  perceptually uniform and accessible
```

12.8 Lab: Comprehensive EDA Dashboard

In this lab you will build a complete exploratory data analysis (EDA) dashboard using a synthetic retail sales dataset. The finished figure will be a multi-panel stakeholder report assembled with GridSpec.

Step 1 — Generate the Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

np.random.seed(42)
n = 1000

df = pd.DataFrame({
    'date'     : pd.date_range('2023-01-01', periods=n, freq='D')[:n],
    'region'   : np.random.choice(['North','South','East','West'], n),
    'category' :
np.random.choice(['Electronics','Clothing','Food','Books'], n),
    'age_group': np.random.choice(['18-25','26-35','36-50','50+'], n),
    'gender'   : np.random.choice(['Male','Female'], n),
    'revenue'  : np.random.exponential(scale=3000, size=n) + 500,
    'units'    : np.random.randint(1, 50, n),
    'discount' : np.random.uniform(0, 0.4, n),
})
df['profit'] = df['revenue'] * (0.3 - df['discount'] + np.random.normal(0,
0.05, n))

print(df.shape)
print(df.head())
```

Step 2 — Sales Trend (Line + Bar)

```
# Aggregate to monthly
df['month'] = df['date'].dt.to_period('M')
monthly = df.groupby('month').agg(revenue=('revenue','sum'),
                                  units=('units','sum')).reset_index()
monthly['month_str'] = monthly['month'].astype(str)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

# Revenue line
ax1.plot(monthly['month_str'], monthly['revenue']/1000,
         color='steelblue', lw=2, marker='o', markersize=5)
ax1.fill_between(range(len(monthly)), monthly['revenue']/1000,
```

```
                  alpha=0.15, color='steelblue')
ax1.set_ylabel('Revenue (₹ 000s)')
ax1.set_title('Monthly Revenue Trend', fontsize=13)
ax1.grid(True, linestyle='--', alpha=0.5)

# Units bar
ax2.bar(monthly['month_str'], monthly['units'],
         color='darkorange', alpha=0.8, edgecolor='white')
ax2.set_ylabel('Units Sold')
ax2.set_title('Monthly Units Sold', fontsize=13)
ax2.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Step 3 — Customer Demographics

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

# Age group histogram
age_order = ['18-25','26-35','36-50','50+']
age_counts = df['age_group'].value_counts().reindex(age_order)
ax1.bar(age_counts.index, age_counts.values,
        color=['#2196F3','#FF9800','#4CAF50','#9C27B0'],
        edgecolor='white')
ax1.set_title('Customer Age Distribution')
ax1.set_ylabel('Count')
for i, v in enumerate(age_counts.values):
    ax1.text(i, v + 5, str(v), ha='center', fontsize=10)

# Gender pie
gender_counts = df['gender'].value_counts()
ax2.pie(gender_counts.values,
        labels   = gender_counts.index,
        autopct  = '%1.1f%%',
        colors   = ['#2196F3','#E91E63'],
        startangle=90,
        explode  = (0.05, 0.05))
ax2.set_title('Gender Split')

plt.suptitle('Customer Demographics', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```

Step 4 — Product Performance (Grouped Bars + Heatmap)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```
# ── Grouped bars: Revenue by Category & Region ──────────
pivot = df.groupby(['region','category'])['revenue'].sum().unstack()
pivot_k = pivot / 1000

x = np.arange(len(pivot_k.index))
w = 0.2
colors = ['#2196F3','#FF9800','#4CAF50','#9C27B0']
for i, (cat, color) in enumerate(zip(pivot_k.columns, colors)):
    ax1.bar(x + i*w, pivot_k[cat], w, label=cat, color=color, alpha=0.85)

ax1.set_xticks(x + w*1.5)
ax1.set_xticklabels(pivot_k.index)
ax1.set_ylabel('Revenue (₹ 000s)')
ax1.set_title('Revenue by Region & Category')
ax1.legend(title='Category')

# ── Heatmap: Avg Revenue by Region × Age Group ──────────
heat_data = df.groupby(['region','age_group'])['revenue'].mean().unstack()
im = ax2.imshow(heat_data.values, cmap='YlOrRd', aspect='auto')
ax2.set_xticks(range(len(heat_data.columns)));
ax2.set_xticklabels(heat_data.columns, rotation=30)
ax2.set_yticks(range(len(heat_data.index)));
ax2.set_yticklabels(heat_data.index)
fig.colorbar(im, ax=ax2, label='Avg Revenue (₹)')
for i in range(heat_data.shape[0]):
    for j in range(heat_data.shape[1]):
        ax2.text(j, i, f'{heat_data.values[i,j]:,.0f}',
                 ha='center', va='center', fontsize=9)
ax2.set_title('Avg Revenue: Region × Age Group')

plt.suptitle('Product Performance Analysis', fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()
```

Step 5 — Assemble the Full Dashboard

```
# Full stakeholder dashboard in one figure
fig = plt.figure(figsize=(18, 14))
gs  = gridspec.GridSpec(3, 3, figure=fig, hspace=0.5, wspace=0.45)

# ── Row 0: Title Banner ──────────────────────────────────
ax_title = fig.add_subplot(gs[0, :])
ax_title.axis('off')
ax_title.text(0.5, 0.6, 'Retail Sales EDA Dashboard — FY 2023',
              ha='center', va='center', fontsize=22, fontweight='bold',
              transform=ax_title.transAxes, color='#1F3864')
ax_title.text(0.5, 0.2, f'Dataset: {len(df):,} transactions | Regions: 4 |
Categories: 4',
              ha='center', va='center', fontsize=12, color='grey',
```

```
            transform=ax_title.transAxes)

# ── Row 1, Col 0-1: Revenue Trend ──────────────────────
ax_trend = fig.add_subplot(gs[1, 0:2])
ax_trend.plot(monthly['month_str'], monthly['revenue']/1000,
            color='steelblue', lw=2.5, marker='o', markersize=5)
ax_trend.fill_between(range(len(monthly)), monthly['revenue']/1000,
alpha=0.15, color='steelblue')
ax_trend.set_title('Monthly Revenue Trend', fontsize=12,
fontweight='bold')
ax_trend.set_ylabel('Revenue (₹ 000s)')
ax_trend.tick_params(axis='x', rotation=45, labelsize=8)
ax_trend.grid(True, linestyle='--', alpha=0.4)

# ── Row 1, Col 2: Gender Pie ───────────────────────────
ax_pie = fig.add_subplot(gs[1, 2])
g = df['gender'].value_counts()
ax_pie.pie(g.values, labels=g.index, autopct='%1.1f%%',
          colors=['#2196F3','#E91E63'], startangle=90)
ax_pie.set_title('Gender Split', fontsize=12, fontweight='bold')

# ── Row 2, Col 0: Age Distribution ─────────────────────
ax_age = fig.add_subplot(gs[2, 0])
age_order = ['18-25','26-35','36-50','50+']
ac = df['age_group'].value_counts().reindex(age_order)
ax_age.bar(ac.index, ac.values,
          color=['#2196F3','#FF9800','#4CAF50','#9C27B0'],
edgecolor='white')
ax_age.set_title('Age Distribution', fontsize=12, fontweight='bold')
ax_age.set_ylabel('Count')

# ── Row 2, Col 1: Category Revenue ─────────────────────
ax_cat = fig.add_subplot(gs[2, 1])
cr = df.groupby('category')['revenue'].sum().sort_values(ascending=True)
ax_cat.barh(cr.index, cr.values/1000, color='steelblue',
edgecolor='white')
ax_cat.set_title('Revenue by Category', fontsize=12, fontweight='bold')
ax_cat.set_xlabel('Revenue (₹ 000s)')

# ── Row 2, Col 2: Profit Heatmap ───────────────────────
ax_heat = fig.add_subplot(gs[2, 2])
hd = df.groupby(['region','category'])['profit'].mean().unstack()
im = ax_heat.imshow(hd.values, cmap='RdYlGn', aspect='auto')
ax_heat.set_xticks(range(hd.shape[1]));
ax_heat.set_xticklabels(hd.columns, rotation=30, fontsize=8)
ax_heat.set_yticks(range(hd.shape[0])); ax_heat.set_yticklabels(hd.index,
fontsize=8)
ax_heat.set_title('Avg Profit Heatmap', fontsize=12, fontweight='bold')
fig.colorbar(im, ax=ax_heat, shrink=0.8)

plt.savefig('eda_dashboard.pdf', dpi=150, bbox_inches='tight')
plt.savefig('eda_dashboard.png', dpi=150, bbox_inches='tight')
plt.show()
print('Dashboard saved!')
```

Quick Reference Cheat Sheet

Task	Code
Create figure + axes	`fig, ax = plt.subplots(figsize=(9, 5))`
Line plot	`ax.plot(x, y, color='steelblue', lw=2, marker='o')`
Scatter plot	`ax.scatter(x, y, s=50, c=z, cmap='viridis', alpha=0.7)`
Bar chart	`ax.bar(categories, values, color='steelblue', edgecolor='white')`
Histogram	`ax.hist(data, bins=20, density=True, edgecolor='white')`
Pie chart	`ax.pie(sizes, labels=labels, autopct='%1.1f%%')`
Box plot	`ax.boxplot(data, patch_artist=True)`
Heatmap	`im = ax.imshow(matrix, cmap='RdYlGn'); fig.colorbar(im, ax=ax)`
3D surface	`ax = fig.add_subplot(111, projection='3d'); ax.plot_surface(X,Y,Z)`
Title / labels	`ax.set_title('T'); ax.set_xlabel('X'); ax.set_ylabel('Y')`
Legend	`ax.legend(loc='upper right', title='Series')`
Grid	`ax.grid(True, linestyle='--', alpha=0.5)`
Remove spines	`ax.spines['top'].set_visible(False)`
Save figure	`fig.savefig('out.pdf', dpi=300, bbox_inches='tight')`
Apply style	`plt.style.use('seaborn-v0_8')`
Multiple subplots	`fig, axes = plt.subplots(2, 2, figsize=(12, 8))`
GridSpec	`gs = gridspec.GridSpec(3,3); ax = fig.add_subplot(gs[0,:])`
Share x-axis	`fig, axes = plt.subplots(3, 1, sharex=True)`
Pandas plot	`df.plot(kind='bar', figsize=(9,5), rot=0)`
Polars → Pandas	`df_pd = df_polars.to_pandas()`
Inline Jupyter	`%matplotlib inline`
Interactive Jupyter	`%matplotlib widget`

End of Module 12

Next: Module 13 — Statistical Analysis with Scipy and Statsmodels

Vikas Karanth U at 2/27/2026 8:51 AM